Organize everything
in one place.

Nodefold

Minimal workspace to manage creative resources

Nodefold

Nodefold is a minimal workspace designed to help you save and organize resources for your creative projects. From images and icons to fonts and web inspiration, it provides a dedicated space to store, structure and keep track of the tools that spark your ideas, whether for immediate use or future reference.

Built around the idea that every resource deserves its own folder, Nodefold allows you to create custom collections and organize everything in a way that fits your workflow.

Nodefold

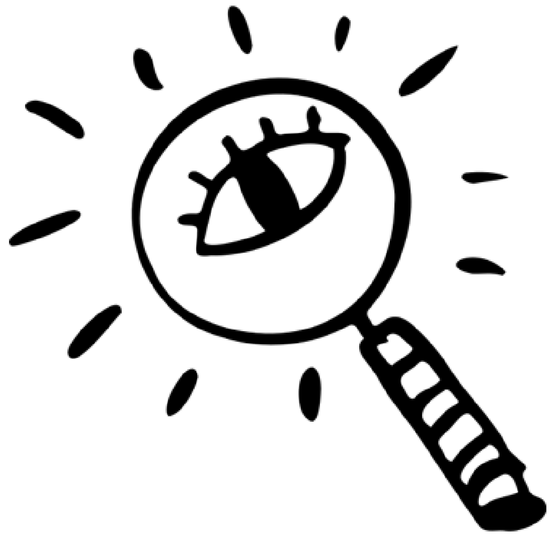
1. Getting ready to work with AI
2. Working with Claude
3. Reviewing Claude’s code
4. Backend & Frontend connection
5. Some thoughts
6. Nodefold

Nodefold

Getting ready to work with AI

Nodefold

Since frontend development in combination with AI was the main focus of this module, I dedicated the first part of the process to researching and selecting the tool that best fit my work style. My selection criteria involved providing each tool with the same project context along with an action plan, and evaluating which one generated the most useful and coherent response. The tools evaluated were ChatGPT, OpenCode, Gemini and Claude, with the latter being chosen for several reasons.



Claude allows projects to be structured with greater precision and depth of context, which is essential for this work, as it is capable of maintaining and retaining the project's central idea regardless of the length of the conversation. Additionally, it supports uploading multiple files without quickly consuming the token limit.

When it needs more information, it generates short questionnaires to refine its response before proceeding. It also performs deeper analyses of what is requested, avoiding repetitive answers that could cause confusion during the process.

To get started on building the frontend, my first interaction with Claude involved providing it with a complete overview of the backend: the list of endpoints, the project's structure and security, the use of Passport (OAuth2) and the fact that it follows RESTful conventions with versioning under /api/v1/.

Although the first prompt was quite extensive, the goal was to ensure it had a full understanding of how the first part of the project was built. This information also included the functional requirements that is what we wanted to achieve and which aspects of development were relevant beyond the simple technical approach.

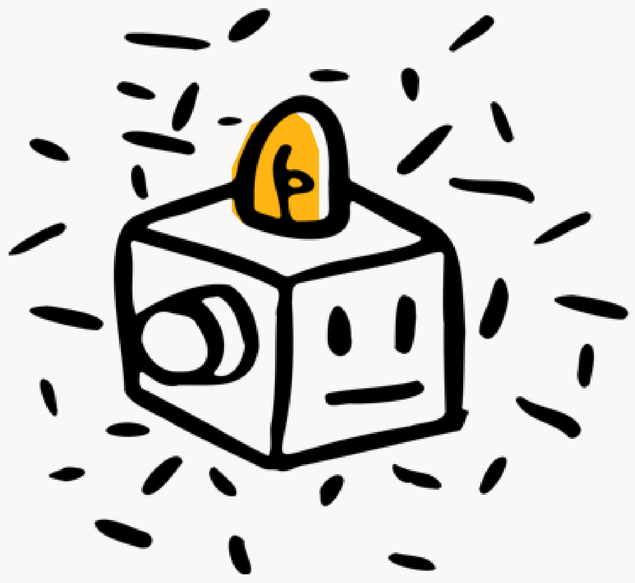
Once the context was established, the initial prompt was:

> ***"Con el conocimiento previo, quiero que generes un plan para estructurar todo el frontend de la API. Antes de escribir cualquier línea de codigo, dame un plan de acción con una estructura de proyecto basada en React. Esta estructura debe ser coherente, organizada y debe respetar las buenas prácticas. En cada paso debes explicar el razonamiento detrás de las decisiones tomadas, así puedo supervisar, preguntar y ajustar cualquier aspecto antes de implementarlo. Cuando sea necesario, incluye referencias visuales para complementar la explicación"***

Nodefold

Working with Claude

Claude set out a series of steps to follow before writing any code, starting with the basics, creating the project and installing the configuration files, along the way, he explained why each step was necessary.



Structure proposed by Claude

- Create the project
- Clean up the project structure
- Install dependencies and set up Git
- Set up React Router and the routing structure
- Context API for authentication
- Set up Axios
- Design system and Theme Context

Step 1 — Create the project
Why Vite and not Create React App?
Why Tailwind CSS?
Run these commands:

Examples of instructions and descriptions

```
npm create vite@latest nodefold-front -- --template react
cd nodefold-front
npm install
```

Next, install **Tailwind**
Then create the Tailwind configuration file in the project root directory

2. Clean up the project structure

```
src/
├── assets/          # Imágenes, iconos, fuentes
├── components/      # Componentes reutilizables (botones, inputs, cards)
├── context/         # Context API para auth y theme
├── hooks/           # Custom hooks
├── pages/           # Una carpeta por vista/página
│   ├── Home/
│   ├── Auth/
│   ├── Dashboard/
│   ├── Folder/
│   ├── Profile/
│   └── Admin/
├── services/        # Llamadas a la API con Axios
├── App.jsx
├── main.jsx
└── index.css
```

Through the entire development process, Claude provided detailed explanations at every stage, which made it easier to understand the code and quickly resolve any issues.

Example of the React Router configuration and route structure.

Why set up the routes now?

React Router defines the navigation for the entire app. If we set it up now with all routes empty, we can then simply fill in each page without changing the navigation structure.

What is a protected route?

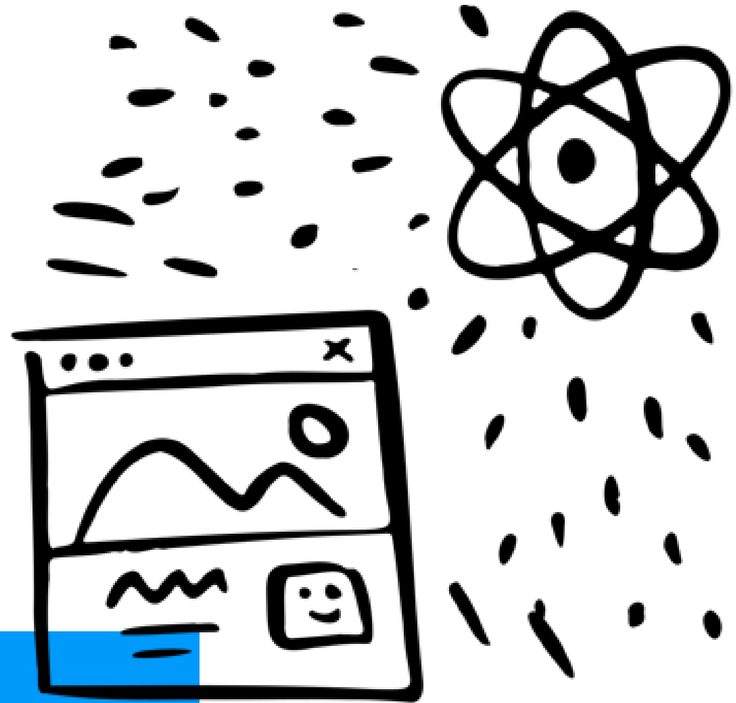
Some pages should only be accessible if the user is authenticated (Dashboard, Profile, Admin). If the user does not have a token, they should be redirected to the login page. This is handled using a ProtectedRoute component.

Create the feature branch:
git checkout -b feat/frontend-setup

Now replace src/App.jsx with this:

```jsx
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'

// Pages (vacías por ahora)
import Home from './pages/Home/Home'
import Login from './pages/Auth/Login'
import Register from './pages/Auth/Register'
import Dashboard from './pages/Dashboard/Dashboard'
import FolderView from './pages/Folder/FolderView'
import Profile from './pages/Profile/Profile'
import Admin from './pages/Admin/Admin'

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Rutas públicas */}
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />

        {/* Rutas protegidas */}
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/folders/:id" element={<FolderView />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/admin" element={<Admin />} />

        {/* Ruta 404 */}
        <Route path="*" element={<Navigate to="/" />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

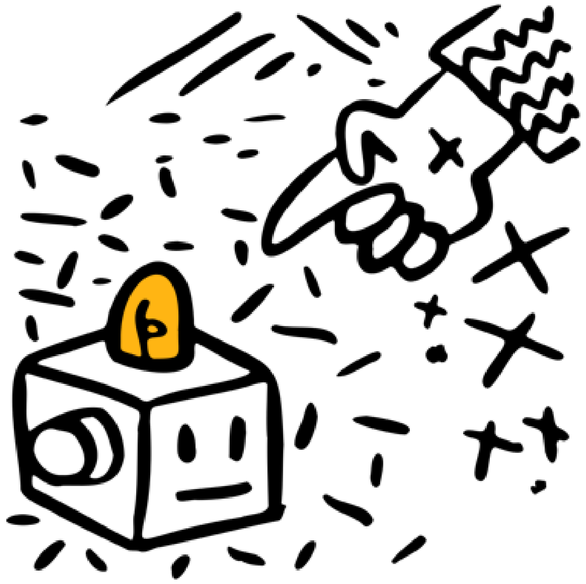
This entire first part allowed us to establish a solid foundation that serves as a starting point for the various sections of the application. Both the project structure and the configuration of the connection to the endpoints using Vite, the bundler used in conjunction with React, set the tone for the work. From there, development proceeded incrementally, starting with the home page and gradually moving on to more complex features.



Nodefold

Reviewing Claude's code

The code generated by Claude was well structured and met the specified requirements. The names were descriptive, React conventions were followed and the layout guidelines were respected. However, the main issue appeared in the separation of concerns within each view, with files exceeding 250 lines that combined logic, styles, and structure elements that could easily have been split into separate components and hooks.



Prompts Examples

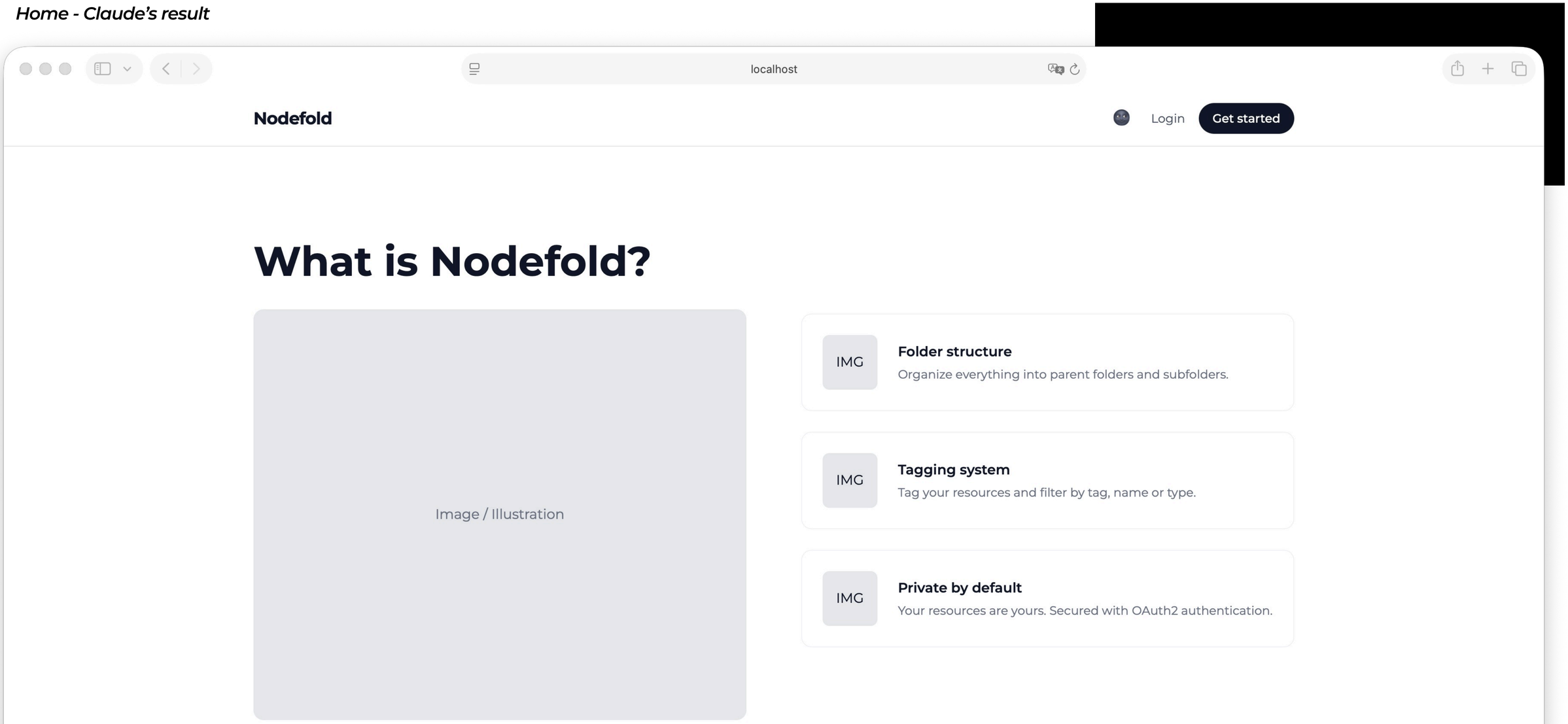
“El home debe estar dividido en 4 partes, la Hero section debe ocupar todo el alto de la pantalla, incluye un header fijo que se mueva al hacer scroll. Debe tener el logo a la izquierda y los botones de login y registro a la derecha, el segundo debe ser redondeado con fondo color negro. La segunda sección debe incluir a la izquierda y centrado un placeholder para una imagen y a la derecha 3 contenedores para características, estos 3 contenedores deben estar organizados uno encima de otro, tener el mismo ancho y alto y cada uno debe incluir un img Holder a la izquierda. Estas tres secciones deben resaltar 3 características de la app”

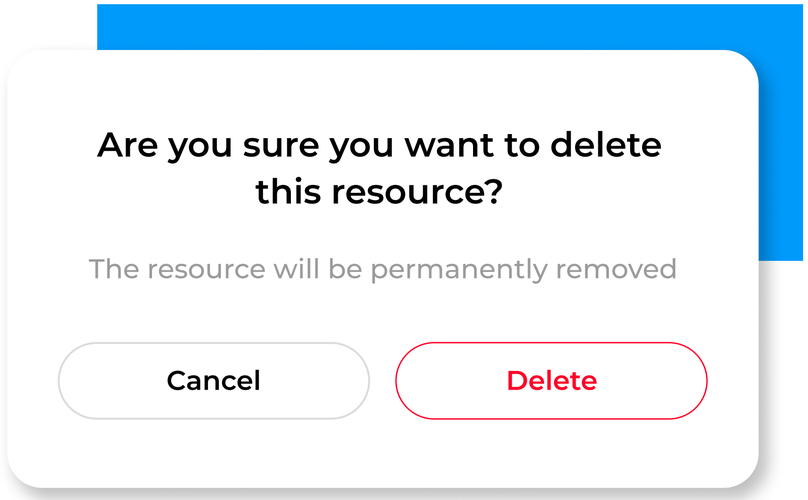
“La barra lateral debe comenzar con un preview del recurso seleccionado seguido del title, description, link, tags y el tipo de recurso que es, en la parte inferior debes incluir dos botones, uno para Edit y otro para Delete. La edición debe activar los inputs dentro de la misma barra, en el modo edición los botones deben cambiar a Cancel y Save”

“La Sidebar debe tener el logo Nodefold en la parte superior del mismo tamaño del home, en la parte de abajo debes poner tres filtros All, Tagged y Untagged y en la parte inferior un botón de ‘+ Create new folder’ todos con un hover redondeado. La parte central del dashboard de momento debe tener un placeholder”

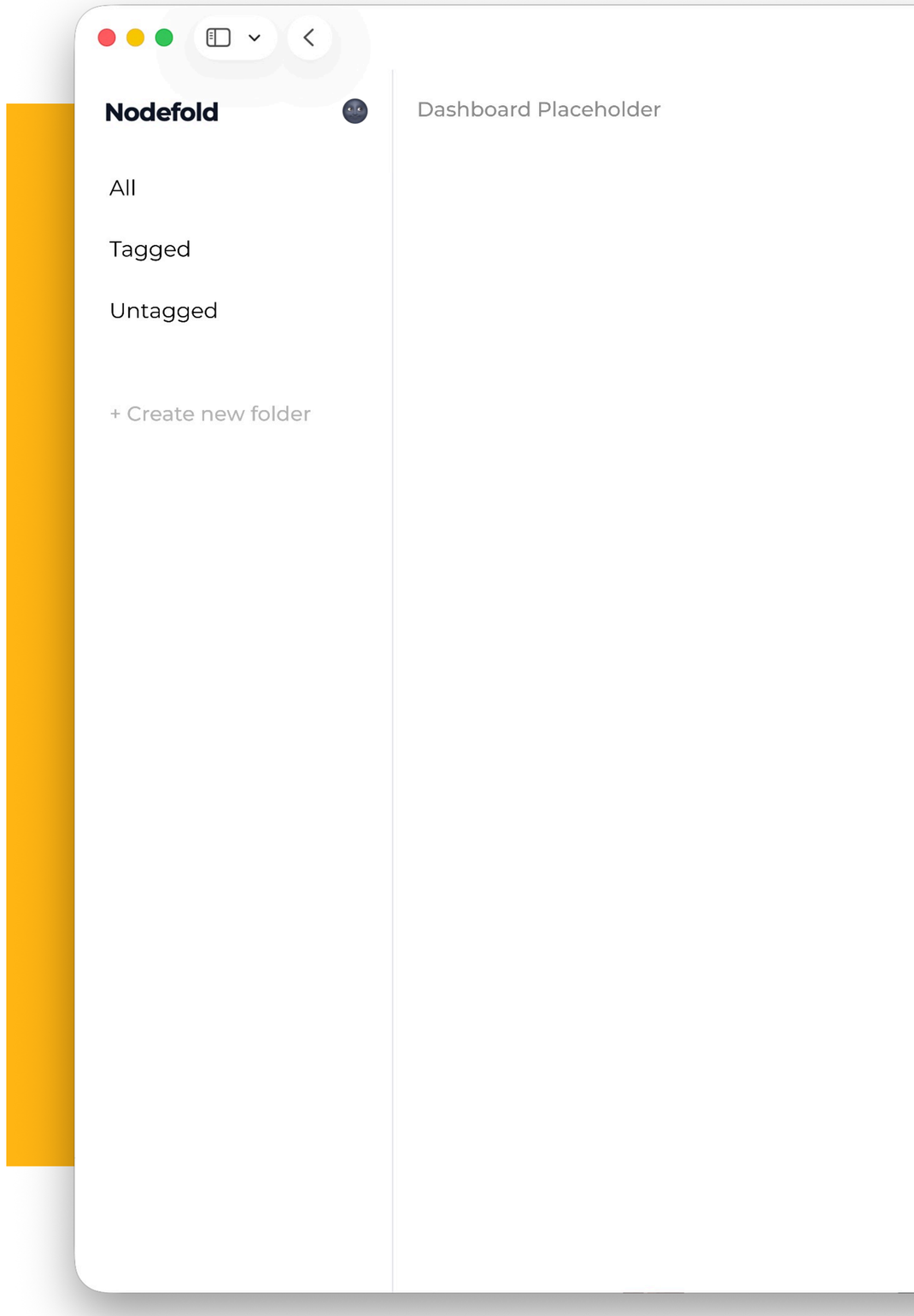
Added to this was a common pattern found in generative AI tools: the tendency to apply default styles when creating layouts. Generic fonts, standard sizes and a recurring use of shadows which, taken together, produce a visually recognizable but undistinct result. For this reason, part of the work involved reviewing and adjusting both the code and the organization of the components to adapt them to the project's visual identity.

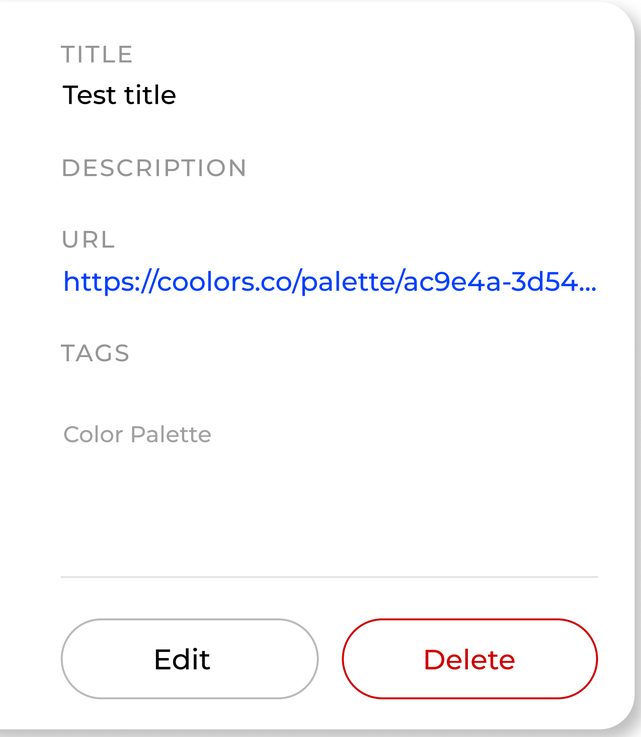
Home - Claude's result





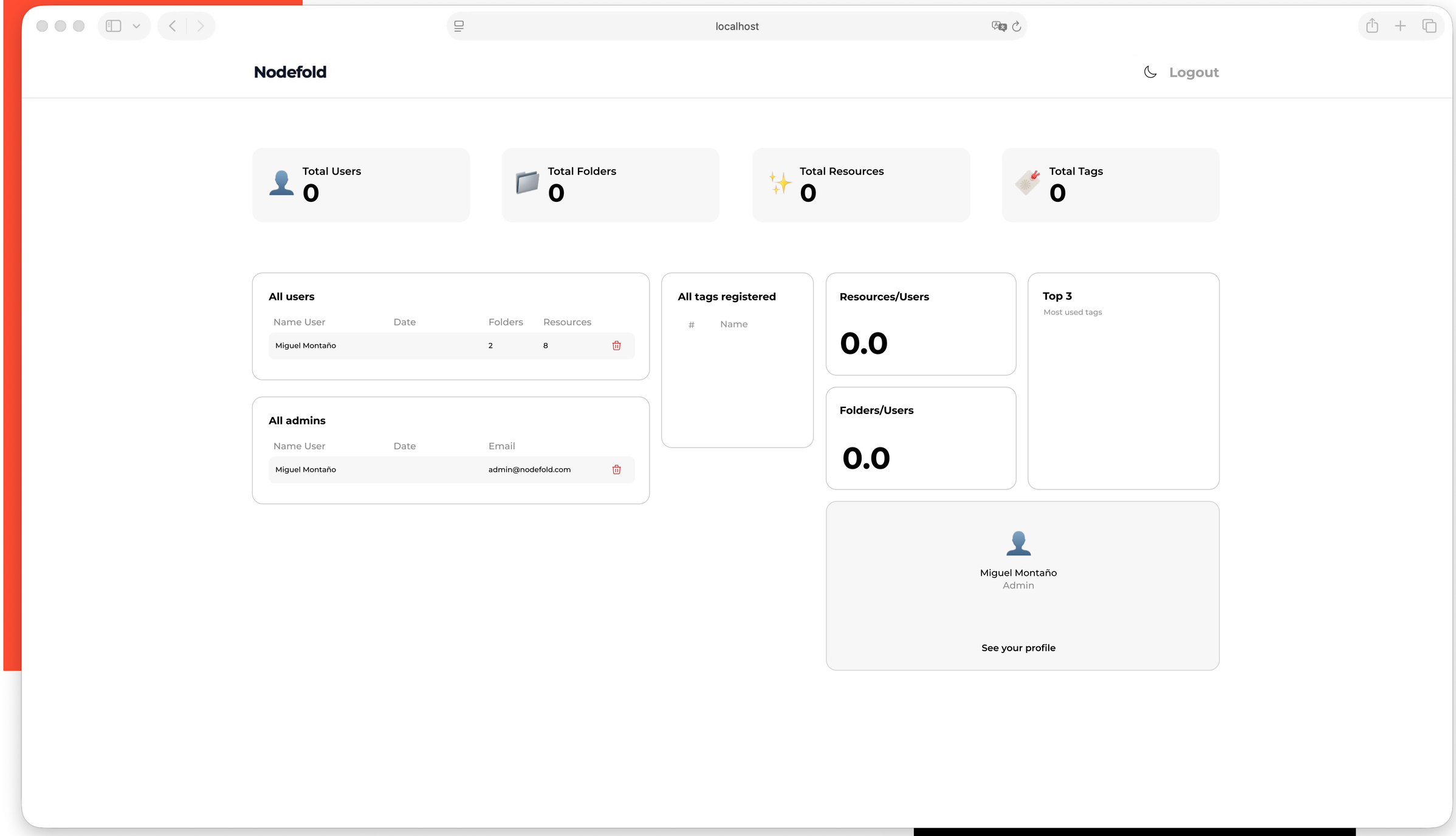
Examples of the views provided by Claude





Despite these changes, the good news was that the proposed logic worked correctly from the start. The proposed solutions met expectations in terms of functionality, which allowed us to focus our review efforts on structure and style without having to rewrite the underlying logic.

Example of the dashboard generated by Claude based on the instructions



Nodefold

Backend & Frontend connection

The General Architecture

Nodefold consists of two separate applications that communicate with each other:

- nodefold-api — Laravel running on **http://localhost:8000**
- nodefold-front — React running on **http://localhost:5173**

The frontend never accesses the database directly. It always communicates with the API via HTTP requests.

The connection point: Axios

In src/services/api.js, we create an Axios instance configured with the API's base URL:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL, // http://localhost:8000/api/v1
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  }
})
  return go(f, seed, [])
}
```

This instance has an interceptor that automatically adds the authentication token to every request:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) {
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})
```

Example — Login

When the user fills out the login form and clicks “Sign In”:

1. The frontend calls the service:

```
// src/services/authService.js
export const login = (data) => api.post('/login', data)
```

2. The Laravel API receives the request in AuthController:

```
public function login(Request $request) {
    if (!Auth::attempt($request->only('email', 'password'))) {
        return response()->json(['message' => 'Invalid credentials'], 401);
    }
    $token = $request->user()->createToken('auth_token')->accessToken;
    return response()->json(['user' => $request->user(), 'token' => $token]);
}
```

3. The frontend stores the token in localStorage and in the AuthContext:

```
const res = await loginService(form)
login(res.data.token, res.data.user) // guarda en localStorage + contexto
navigate('/dashboard')
```

From now on, all requests automatically include the token thanks to the interceptor.

Connection Flow Recap

User interacts with React

↓

Axios service makes an HTTP request with a Bearer token

↓

Laravel validates the token with Passport

↓

Controller processes the logic and queries the database

↓

Returns JSON to the frontend

↓

React updates the state → the UI is re-rendered

Nodefold

Challenges met

Uploading images, but the API didn't receive the data

By default, Axios sends requests as JSON. But to upload files, we need multipart/form-data. If we mix the two, the API won't receive either the file or the data correctly.

The solution: create a FormData

src/pages/Dashboard/components/AddResourceModal.jsx

```
const handleSubmit = async (e) => {
  const data = new FormData()
  data.append('type', form.type)
  data.append('title', form.title)
  data.append('url', form.url)
  data.append('image', imageFile) // archivo real del input

  await createResource(folderId, data)
}
```

Request blocked by the browser

When React on localhost:5173 tries to make a request to Laravel on localhost:8000, the browser blocks the request because they are different origins.

The solution in Laravel: config/cors.php

config/cors.php

```
return [
    'paths' => ['api/*', 'docs*', 'sanctum/csrf-cookie'],
    'allowed_methods' => ['*'],
    'allowed_origins' => ['*'],
    'allowed_origins_patterns' => [],
    'allowed_headers' => ['*'],
    'exposed_headers' => [],
    'max_age' => 0,
    'supports_credentials' => false,
```

Editing the folder name

When triggering the event to rename a folder, the form would automatically submit, making it impossible to perform the action smoothly.

The solution: Wrap the input in a
<form onSubmit={onCreate}>

.../components/Sidebar/components CreateFolderInput.jsx

```
<form onSubmit={onCreate} className="mt-2 px-1">
    <input
      type="text"
      value={newFolderName}
      onChange={(e) => setNewFolderName(e.target.value)}
      onKeyDown={(e) => {
        if (e.key === "Escape") {
          setShowInput(false);
          setNewFolderName("");
        }
      }}
      onBlur={() => {
        setShowInput(false);
        setNewFolderName("");
      }}
      placeholder="Folder name"
      autoFocus
      className="w-full px-3 py-2 text-sm rounded-lg
border border-gray-200 dark:border-gray-700 bg-white
  dark:bg-gray-900 focus:outline-none focus:ring-2
focus:ring-gray-900 dark:focus:ring-white"
   />
  </form>
```

Generally, the problems that came up during development were isolated and in most cases, solutions were found right away. It’s key to note that these issues weren’t related to the code structure itself, but rather to specific behavioral details that I, as the developer, was requesting. In many cases, it was the requests themselves that introduced bugs or inconsistencies, whether due to a lack of precision or conflicting requirements. A subsequent review was sufficient to identify and resolve them.

Replay of the tour after it ends

Once the tour was completed, it would restart from the beginning if the page was reloaded. The localStorage key was shared, which meant that regardless of whether the tour was completed or canceled midway, it would start over.

The solution: One key per user

src/pages/Dashboard/hooks/useTour.js

```
import { useState } from "react";
 import { useAuth } from "../../../context/AuthContext";

export function useTour() {
  const { user } = useAuth();
  const TOUR_KEY = `nodefold_tour_done_${user?.id}`;
  const [tourActive, setTourActive] = useState(
    () => localStorage.getItem(TOUR_KEY) !== "true",
  );
  const handleTourEnd = () => {
    localStorage.setItem(TOUR_KEY, "true");
    setTourActive(false);
  };
  const restartTour = () => {
    localStorage.removeItem(TOUR_KEY);
    setTourActive(true);
  };
  return { tourActive, handleTourEnd, restartTour };
}
```

Nodefold

Some thoughts

Although I began this phase of the project with no prior knowledge of React or the connection between frontend and backend, the ability to outline the project's full scope for the AI and establish a clear workflow allowed me to gradually absorb terms, structures and logic that had previously been unfamiliar to me. The learning process was not passive: understanding each decision before implementing it was what made all the difference.

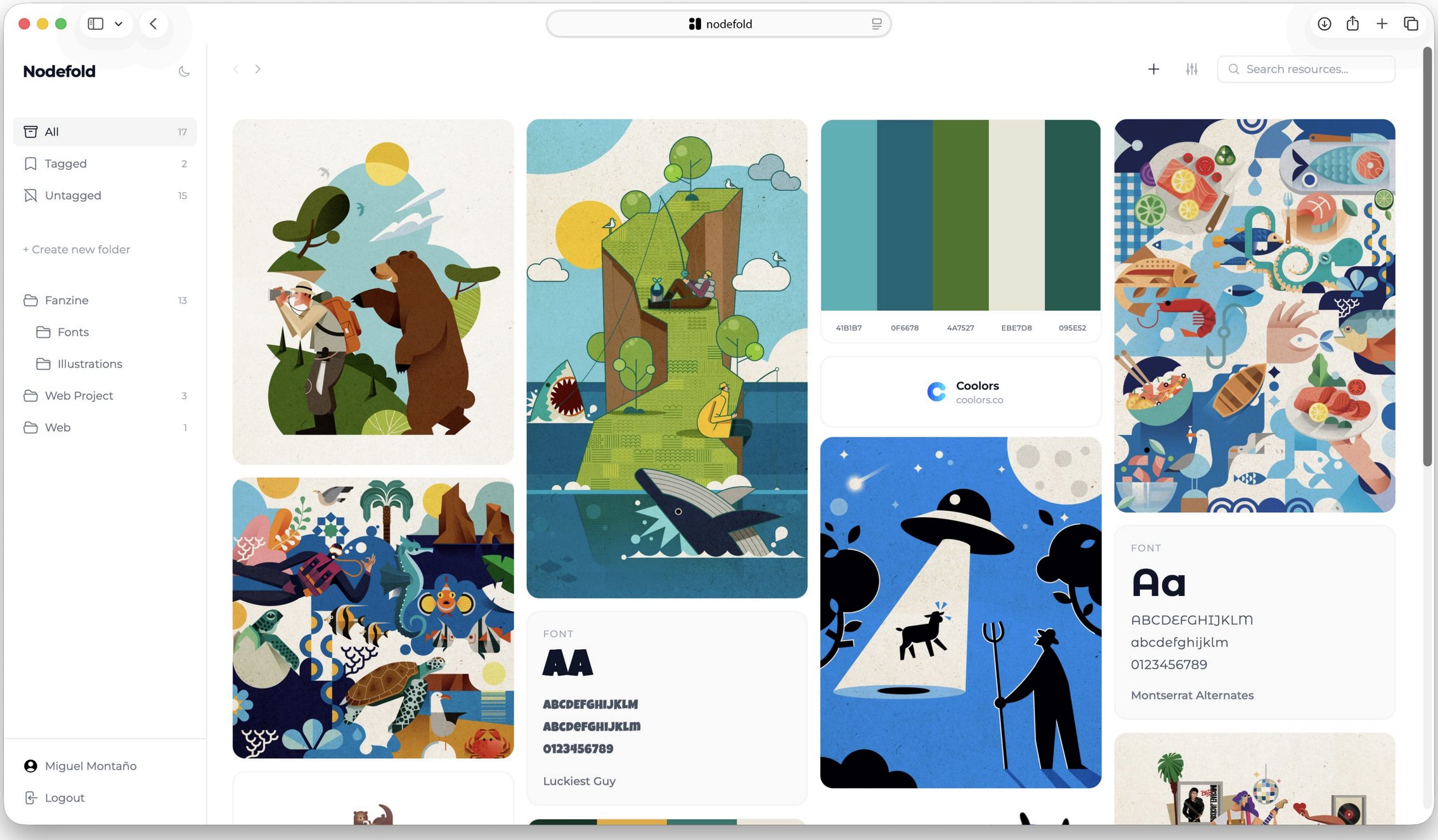
Personally, the most interesting part of this stage was shaping an idea that already existed and that I had worked on in a different way, but incorporating features that, had they been researched independently, would have required twice as much time. However, the real challenge wasn't in fixing bugs or connecting the different parts, but in understanding the reasoning behind each decision: understanding the logic behind the code and not simply applying it.

Having worked on the API independently first and the frontend later allowed me to understand, on the one hand, how an API should be properly structured and on the other, how to build from the frontend with specific functionalities in mind and moving forward progressively. The autonomy gained throughout this process is, without a doubt, the most valuable lesson and the one that will be reflected in future projects.

Nodefold

Minimal workspace to manage creative resources

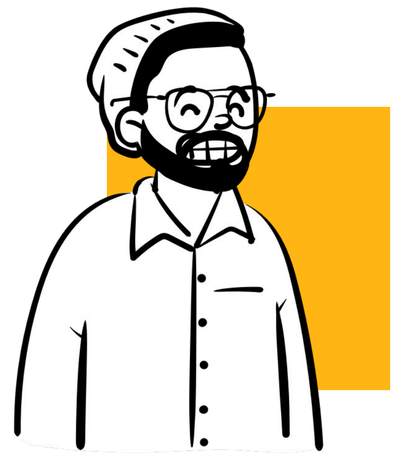
2026



See the live demo here

https://nodefold.vercel.app

Nodefold



Miguel Montaño

GitHub

Nodefold-API
Nodefold-Front